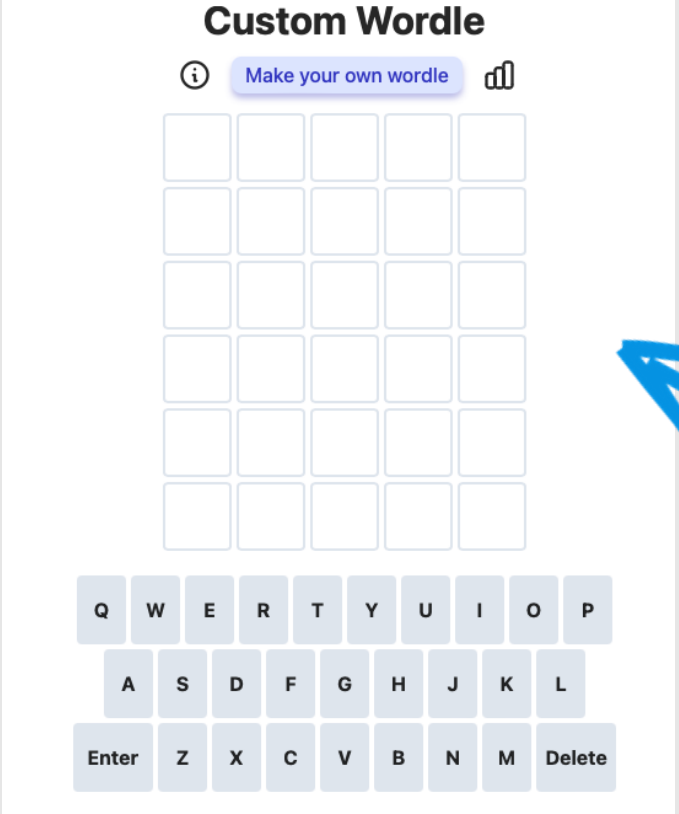


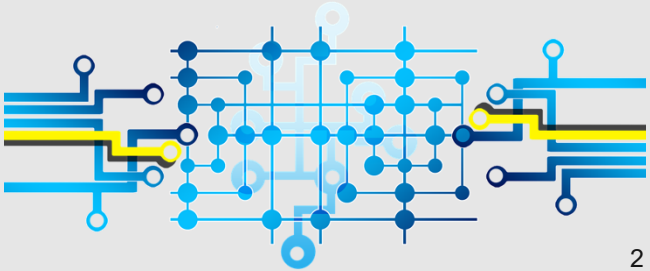
Play the custom wordle (guess the word within 6 guesses) – hint, it's generative development related!

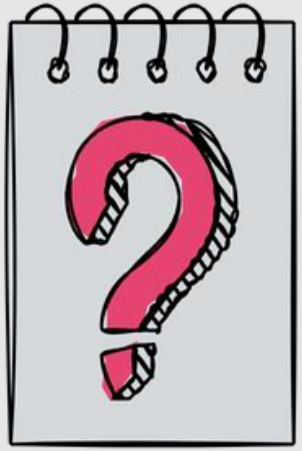
https://bit.ly/gd-wordle

CO4207 / CO7207 Generative Development

Lecture 4: Class Diagrams

Dr James Hoey *(he/him)*
jbh13@leicester.ac.uk





Intended Learning Outcomes…

By the end of today, you will be able to:

- Describe what a class diagram is, and how we use it
- Explain different types of relationships between classes
- Understand common inheritance problems

Have you created Class Diagrams before?

Set up the options. Choose one or multiple correct answers.

Number of student responses

Yes, I feel very confident in creating and reading them	Correct answer	15
Yes, I feel somewhat confident in creating and reading them	Correct answer	32
Yes, but I do not feel confident in creating and reading them	Correct answer	21
No, I have never seen them before	Correct answer	6

Quick Recap

- Can you remember back to Friday?

CO7207 Jan 2026

Visit **app.tophat.com/access**

Code **296607**

Which model would you create to…

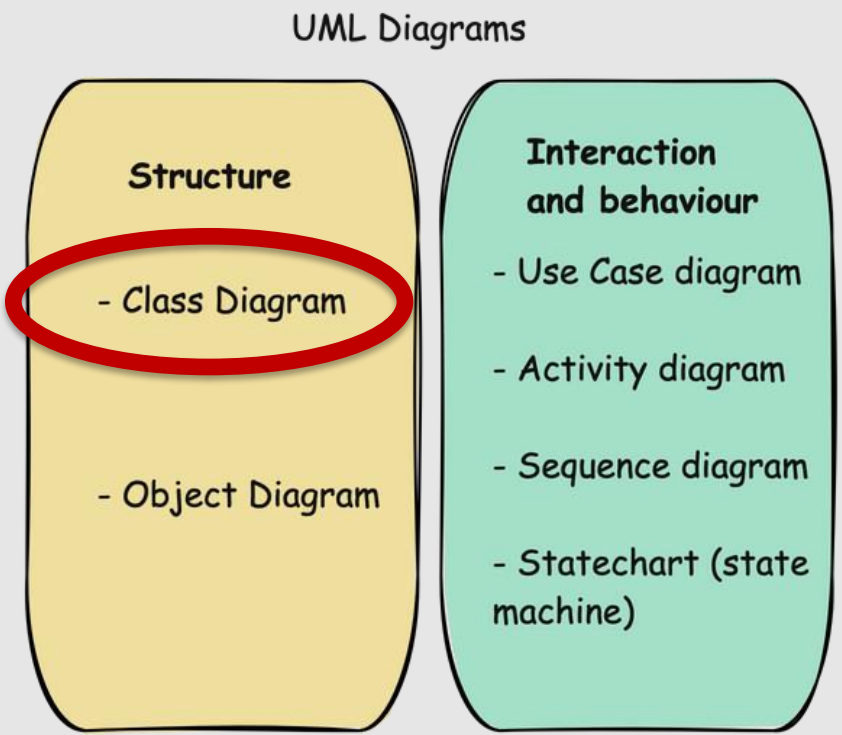
- Model the different ways a user can interact with the control system of our Nuclear Reactor?

Use Case Model

- Model the elements of our Nuclear Reactor and the data structures that the admin system will store?

Need something that models Structure

Today


UML Diagrams
Structure
- Class Diagram
- Object Diagram
Interaction and behaviour
- Use Case diagram
- Activity diagram
- Sequence diagram
- Statechart (state machine)

Overview for today…

- Classes
- Associations
  - Composition / Aggregation / Navigability / Association Class

- Generalisation
  - Single inheritance
  - Multiple inheritance

Class Diagrams

- Model from UML, that can express
  - Classes
  - Attributes and methods
  - Relationships between classes

Point
-x : double -y : double
+getX () : double +getY() : double +move(dx : double, dy : double) # void rotate()

- Structure! (use cases capture behaviour)
- Can formalise OO-systems, data models, etc.

Classes

Class Name

Point
-x : double -y : double
+getX () : double +getY() : double +move(dx : double, dy : double) # void rotate()

Attributes

(Visibility name : type)

visibility is + (public), - (private), # (protected)

Methods

(Visibility name (parameters): return type)

Where visibility is + (public), - (private), # (protected)

Course Scheduling System

CloseRegistrationController

+ isRegistrationOpen()
+ closeRegistration()

Student

+ getTuition()
+ addSchedule()
+ getSchedule()
+ deleteSchedule()
+ hasPreRequisites()

Professor

- name
- employeeID : UniqueId
- hireDate
- status
- discipline
- maxLoad

+ submitFinalGrade()
+ acceptCourseOffering()
+ setMaxLoad()
+ takeSabbatical()
+ teachClass()

Schedule

- semester

+ commit()
+ selectAlternate()
+ removeOffering()
+ level()
+ cancel()
+ getCost()
+ delete()
+ submit()
+ save()
+ anyConflicts()
+ createWithOfferings()
+ updateWithNewSelections()

Class Diagrams

- Our Nuclear Reactor have a serial number and a date of the last safety check

- How could we store this?

Reactor
-serial : String -dateOfLastCheck : Date

Stored using attributes!

Modified using operations!

Attributes

[visibility] [/] name [: type] [multiplicity] [=value] [{property}]

- Anything in [ ] is optional
- visibility
  - + (public), # (protected), ~ (package), - (private)
- / means derived (e.g. “full name” from “first” + “last”)
- Multiplicity, by default 1
- property (e.g. readOnly, ordered, etc)
- static attributes are underlined

Point
-x : double -y : double
+getX () : double +getY() : double +move(dx : double, dy : double) # void rotate()

Operations

[visibility] name ([parameter-list]) [: type] [{property}]

Point
-x : double -y : double
+getX () : double +getY() : double +move(dx : double, dy : double) # void rotate()

- Anything in [ ] is optional
- Visibility as for attributes (previous slide)

- Method name
- Parameter list (separated by commas, name : type)
- Static operations are underlined

- For example, display(), - hide(), + add(x: int, y: int)

Class Diagrams

Reactor
-serial : String -dateOfLastCheck : Date

- Each **Nuclear Reactor** is maintained by trained **engineers** (that maintain specific parts).

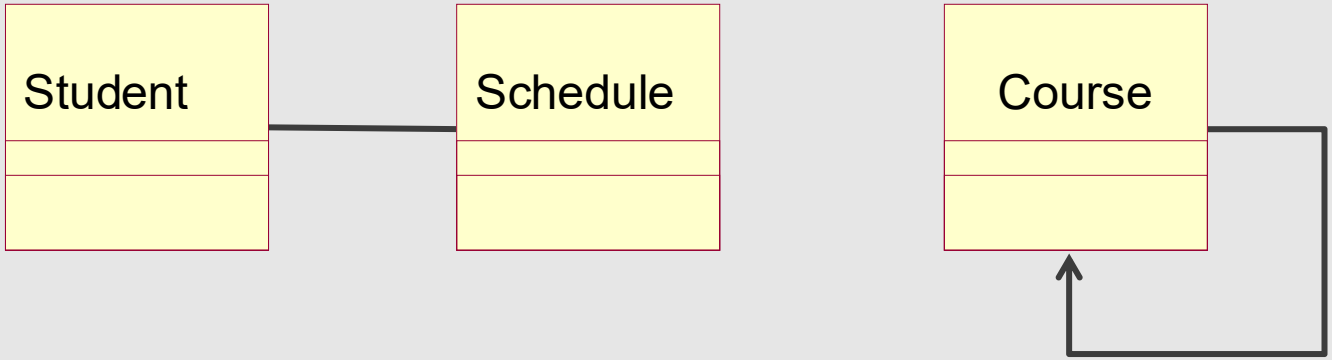
- How could we model this?

Classes

Using relationships between classes!

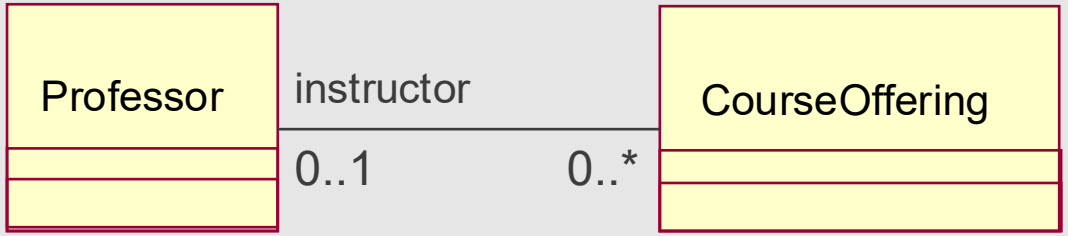
Associations

- The semantic relationship between two or more classes that specifics the connections between their instances
- A structural relationship specifying that objects of one thing are connected to objects of another thing



Associations: Multiplicities

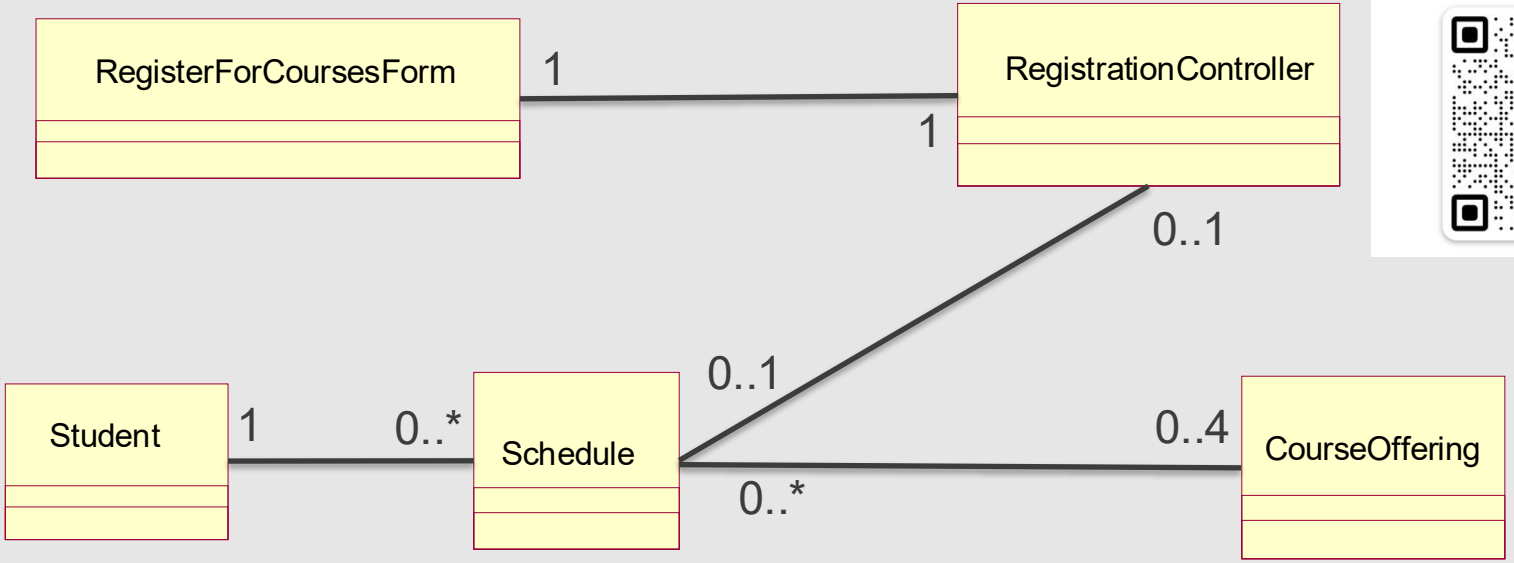
- A multiplicity is the number of instances of one class that relates to (connected to) to ONE instance of another class
- Each association has two multiplicity decisions – one for each end
  - For each instance of Professor, many CourseOfferings can be taught
  - For each instance of CourseOffering, there may be either one or zero Professors as the instructor



Associations: Multiplicities

Unspecified	
Exactly one	1
Zero or more	0 .. *
Zero or more	*
One or more	1 .. *
Zero or One	0 .. 1
Specified range	2 .. 4
Multiple, disjoint ranges	2, 4 .. 6

Associations: Multiplicities



CO7207 Jan 2026
Visit **app.tophat.com/access**
Code **296607**

Associations: Multiplicities

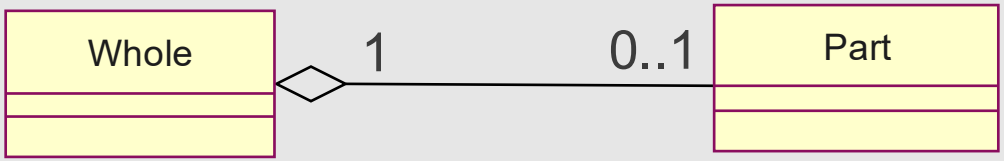
- But what if we need stronger constraints.

- For example, a Nuclear Reactor contains several graphite moderators. Each moderator can never be used with any other reactor (or without a reactor).

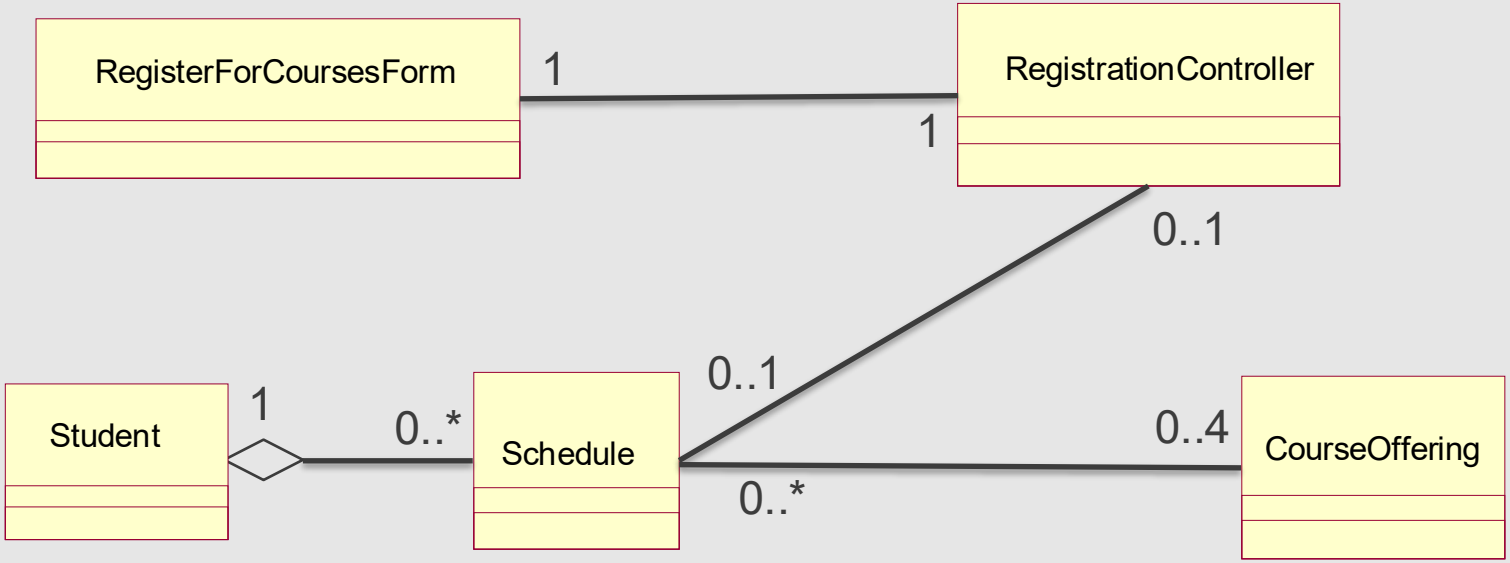
Association isn’t strong enough

Aggregation

- A form of association that models a “whole-part” relationship between the aggregate (the whole) and its parts
  - Is an “is a part of” relationship
  - With multiplicities as before

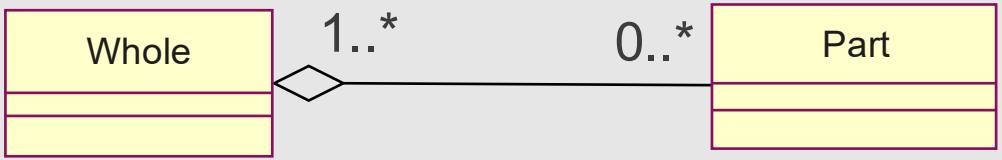


Aggregation: Example

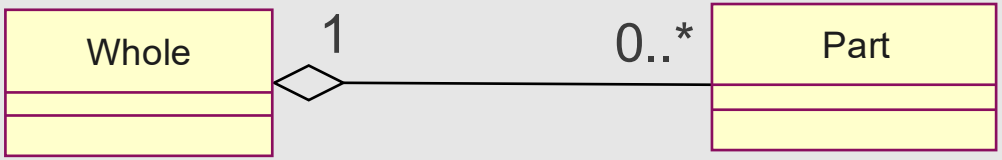


Shared vs Non-Shared Aggregation

- Shared Aggregation = parts can be shared by wholes

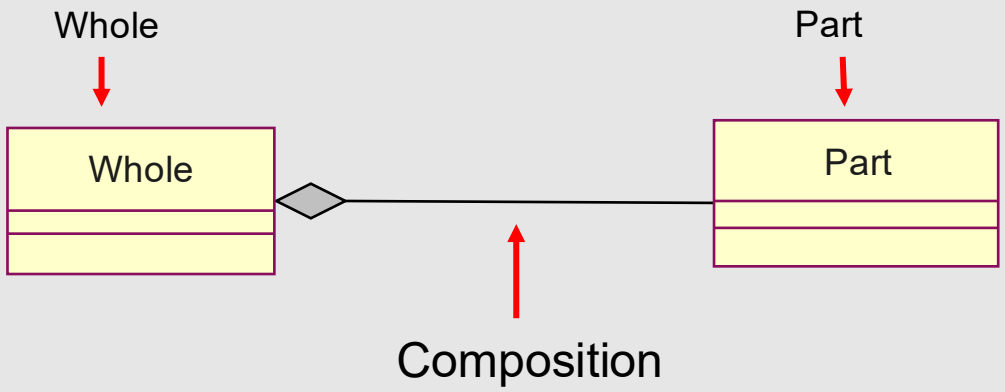


- Non-shared aggregation = parts cannot be shared by wholes



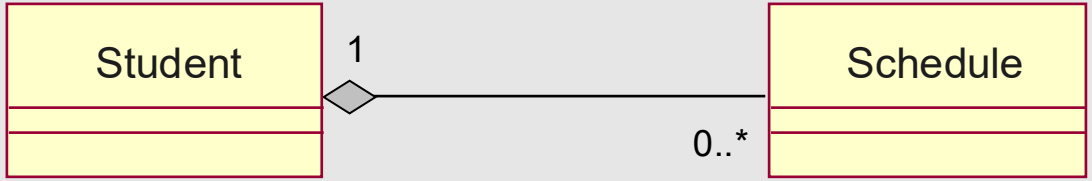
Composition

- A form of association with strong ownership and lifetimes
  - Implementing “non shared aggregation”
  - The “part” cannot survive without the “whole”



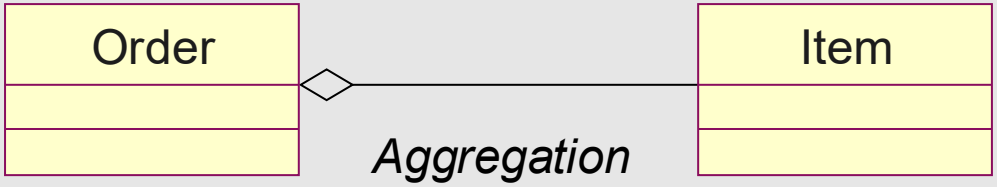
Composition Example

- The “Schedule” can’t exist without a “Student”



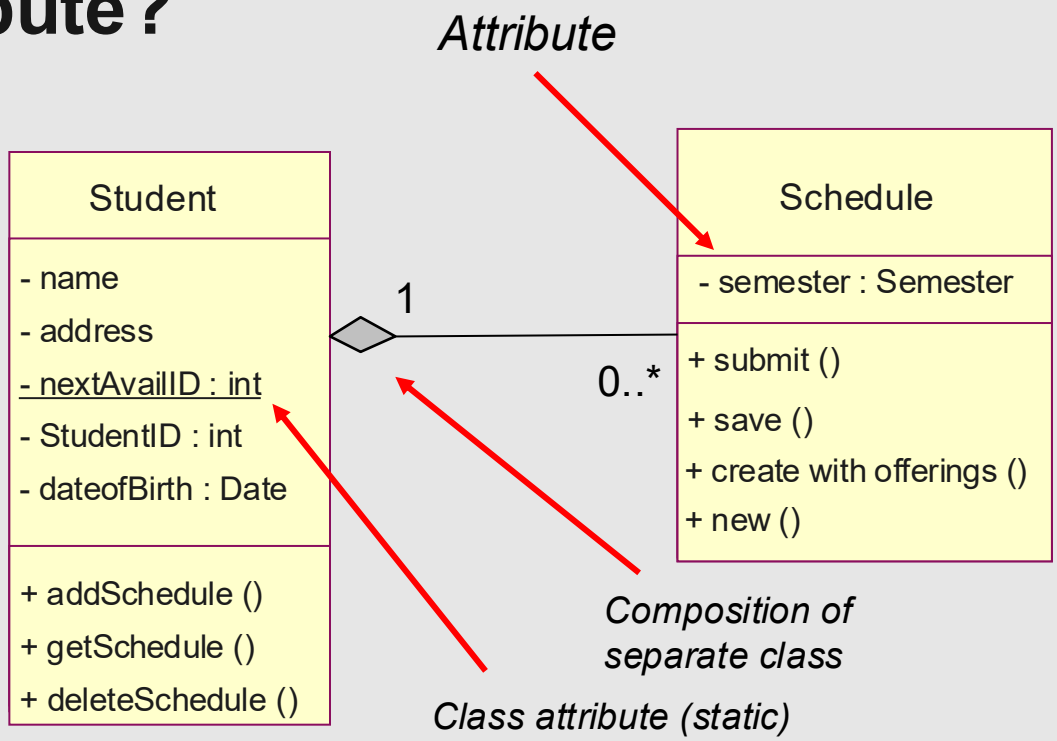
Aggregation vs Composition

- Think about lifetimes



Association or Attribute?

- Use an association when:
  - Properties need independent identities
  - Properties have a complex structure and properties of their own
  - Properties have complex behaviour of their own
  - Properties have relationships of their own
- Otherwise use attributes



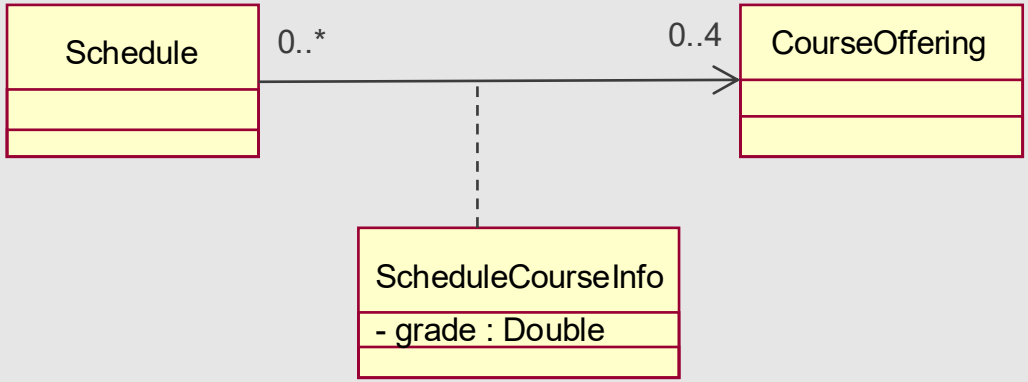
Navigability

- Arrow that indicates it is possible to “navigate” from an associating class to the target class; using the association

- Employees can see which reactors they can/will service.

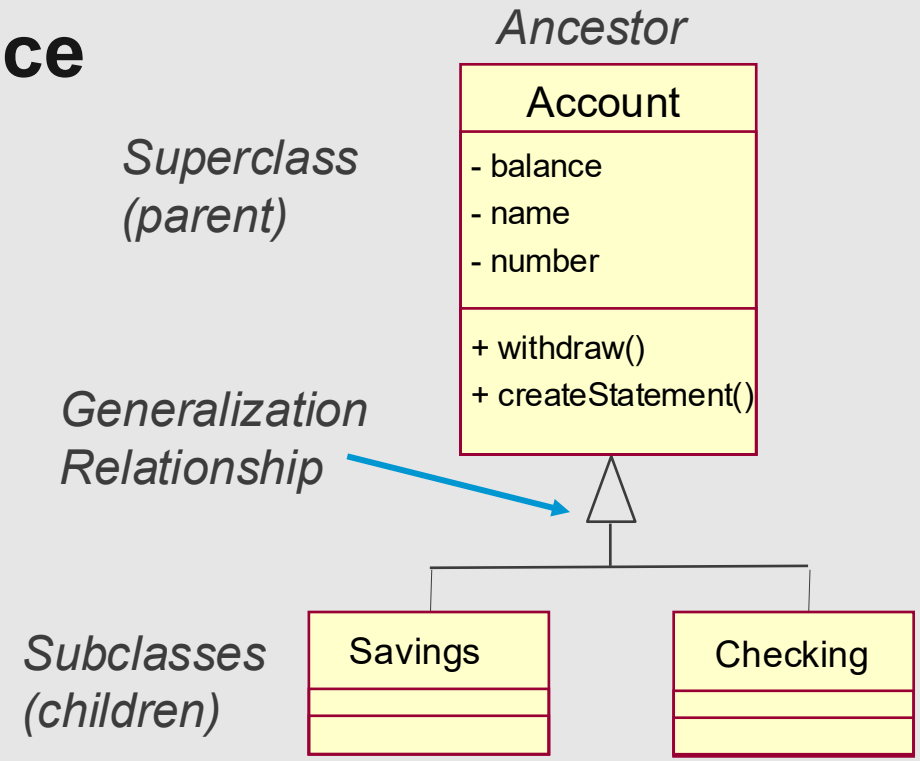
Association Class

- A class that is “attached” to an association
- Contains properties of the relationship
- Has one instance per link



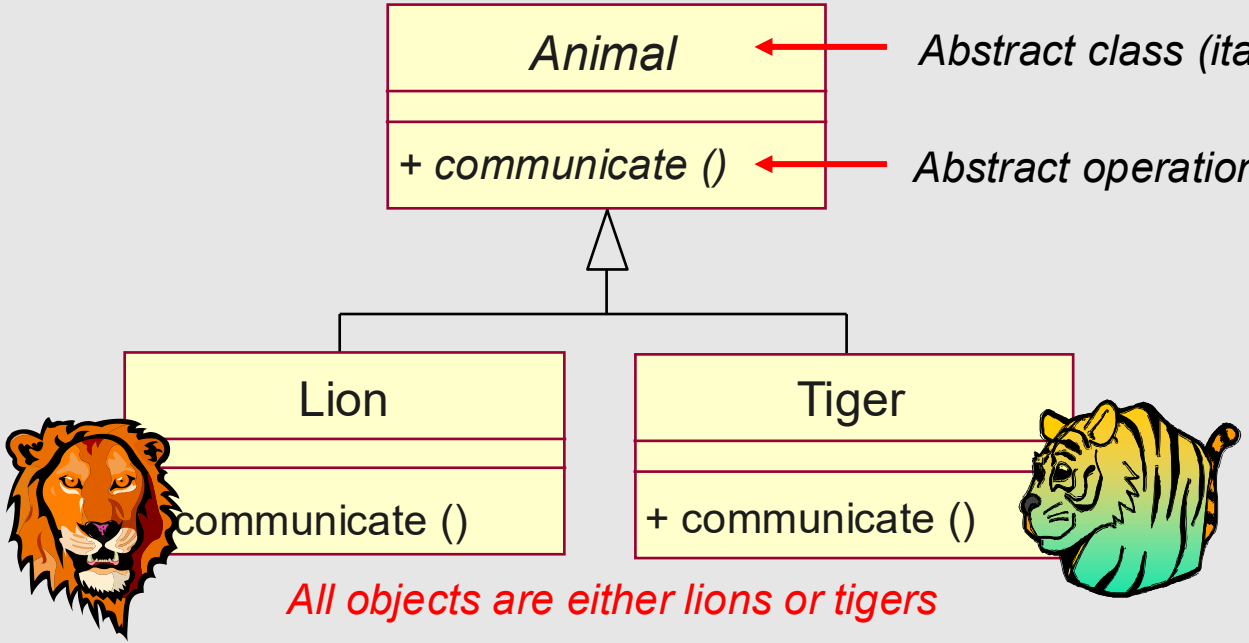
Other links? Inheritance

- A relationship between classes where one class shares the structure and/or behaviour of the other
- Represents a “is a kind of” relationship



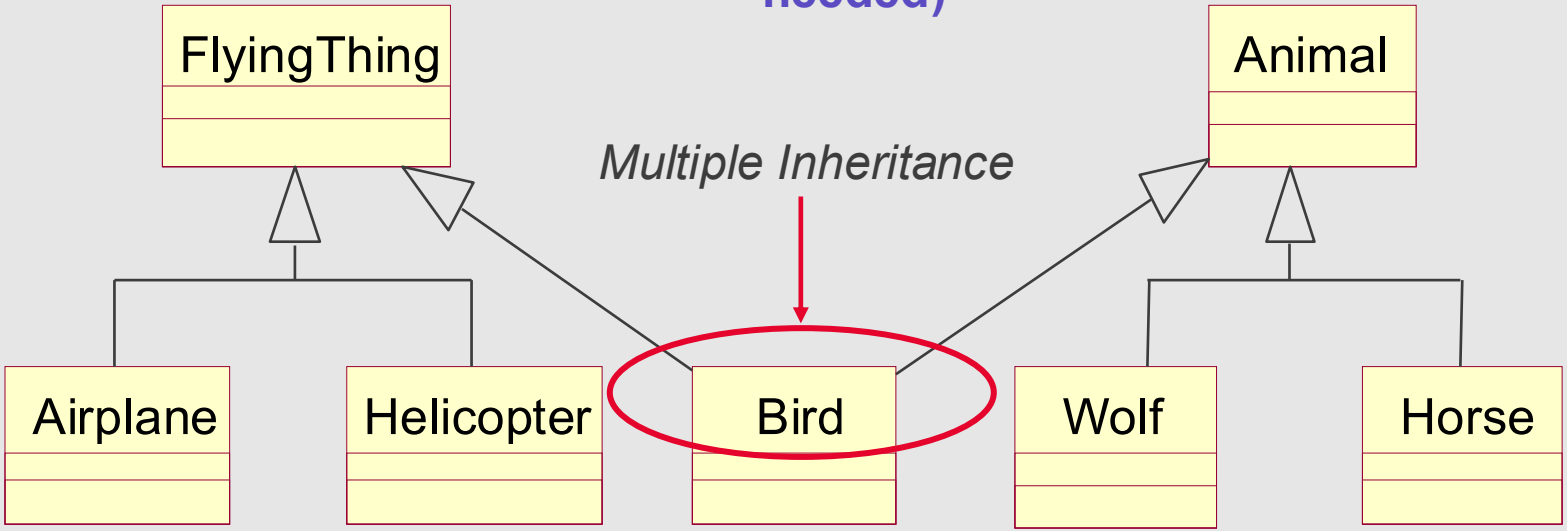
Inheritance Example

- Abstract classes – can't have an object of it
- Concrete classes – can have object of it



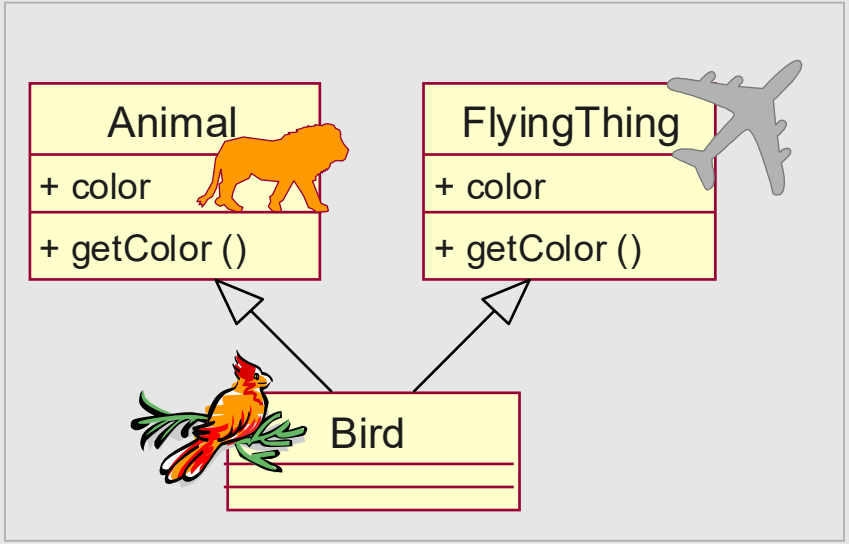
Multiple Inheritance

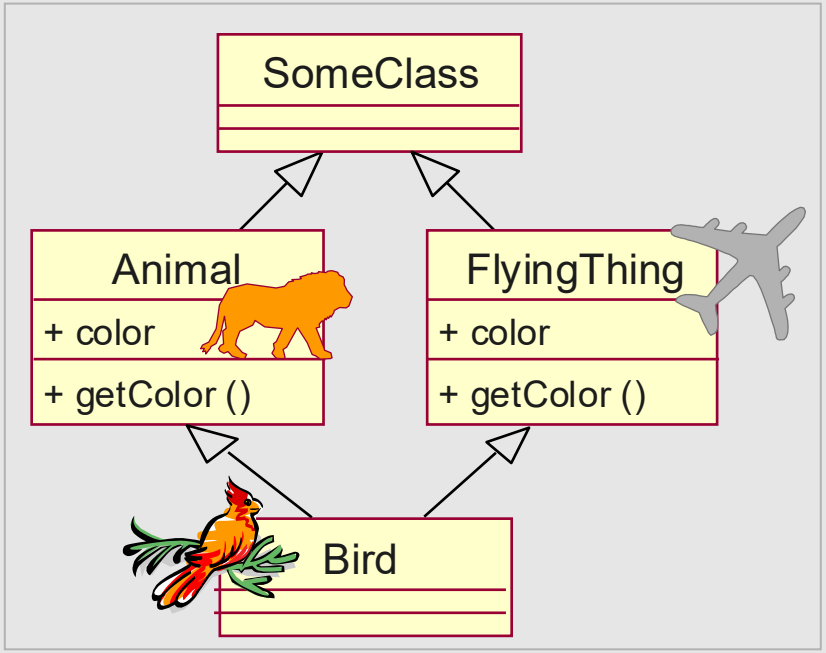
- A class can inherit from several classes
- **Use with caution! (and only when needed)**



Multiple Inheritance Problems

- Name clashes on attributes/operations and Repeated inheritance





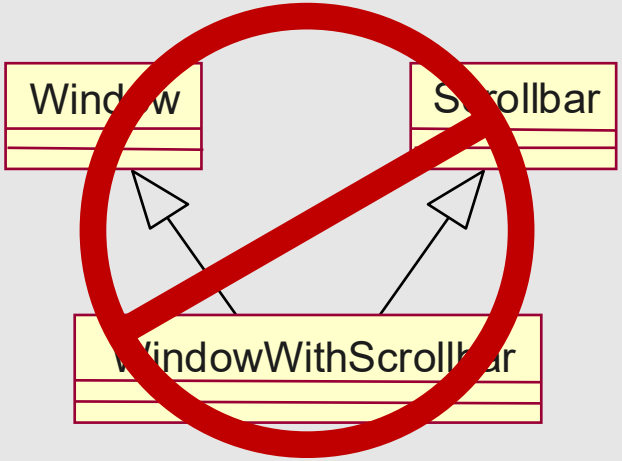
(If you're interested -) Restricting Inheritance

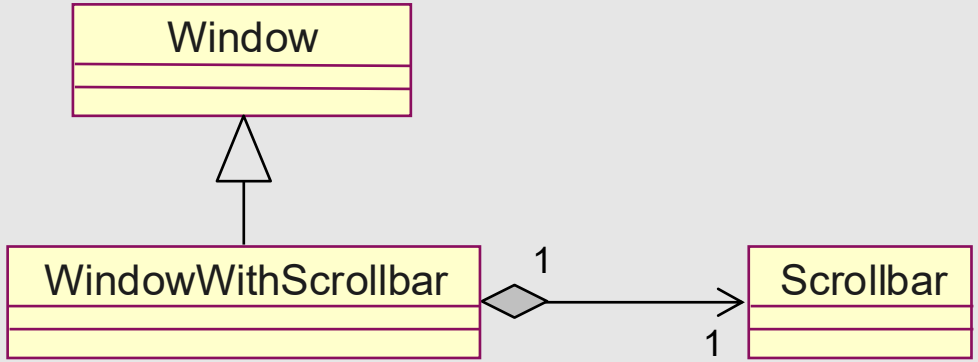
- Generalisation constraints can be added to a class diagram to limit this
- https://www.uml-diagrams.org/generalization.html

- **Complete** = end of the inheritance tree (all children are specified)
- **Incomplete** = inheritance tree may be extended

- **Disjoint** (subclasses mutually exclusive, and doesn't support multiple inheritance)
- **Overlapping** (subclasses not mutually exclusive, and does support multiple inheritance)

Inheritance vs Aggregation





A WindowWithScrollbar "is a" Window
A WindowWithScrollbar "contains a" Scrollbar

Summary and Next time…

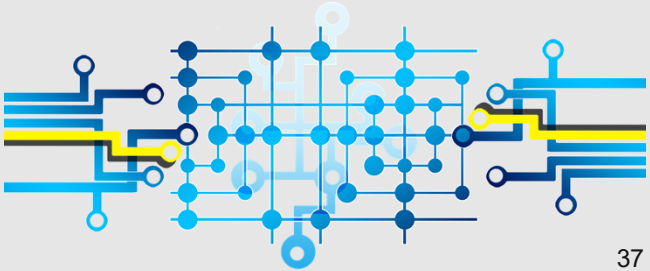
- Describe what a class diagram is, and how we use it
- Explain different types of relationships between classes
- Understand common inheritance problems

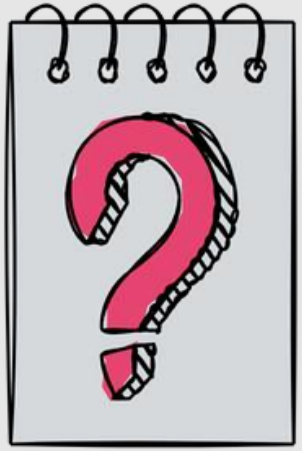
Next time: Object Diagrams

CO4207 / CO7207 Generative Development

Lecture 5: Object Diagrams

Dr James Hoey *(he/him)*
jbh13@leicester.ac.uk



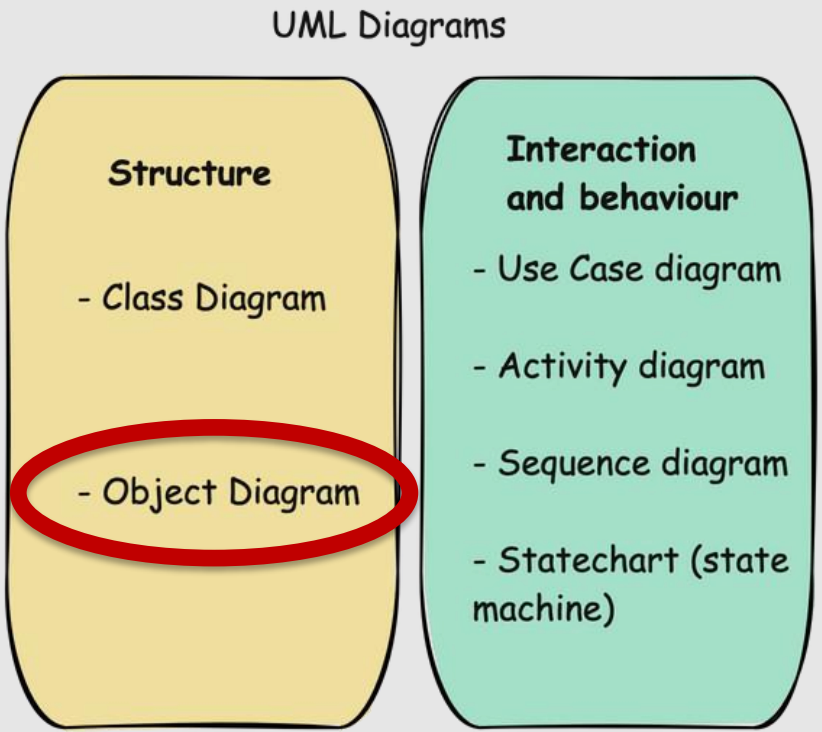


Intended Learning Outcomes…

By the end of today, you will be able to:

- Describe what an object diagram is, and how we use it
- Explain how to create an object diagram from a corresponding class diagram
- Understand what “consistency” means in this context

Today


UML Diagrams
Structure
- Class Diagram
- Object Diagram
Interaction and behaviour
- Use Case diagram
- Activity diagram
- Sequence diagram
- Statechart (state machine)

Class Diagram for our reactor

- “The Springfield power plant (Nuclear Reactor) has two reactors, each with 3 and 5 control rods, respectively. Homer maintains both of these reactors”

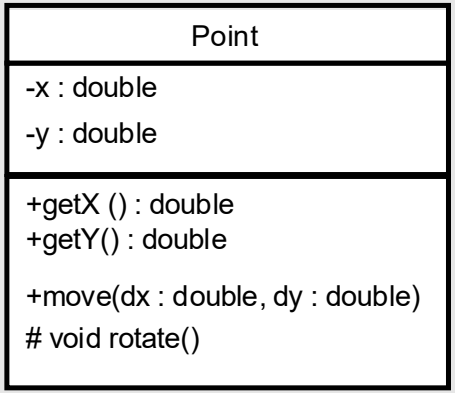
Class diagrams capture all reactors, we need something else for specific instances

Object Diagrams

- Represents **a set of objects** that are instances of the classes defined in the class diagram
- And captures a "**system snapshot**" (state)
- Specifying the values of object **attributes and associations** (links) from a specific moment in time

- Each point in time corresponds to an object diagram (snapshot)

Example: Object Diagram



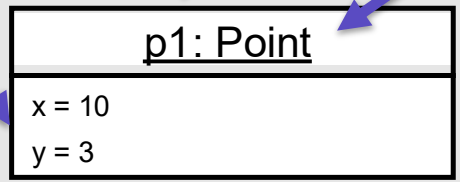
Class diagram

Name of a concrete class (is optional)

: Class name (underlined) (mandatory – including the :)

Attributes with current values

Object diagram

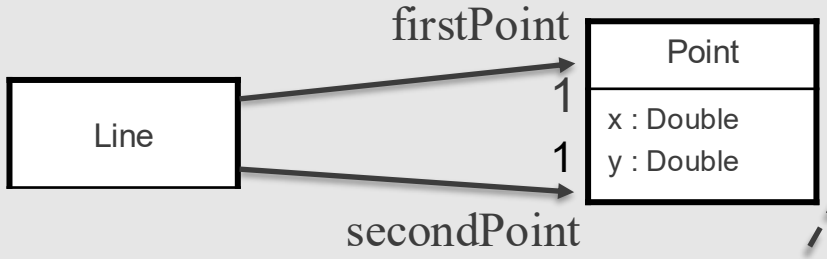


Associations and Aggregations

- Classes are instantiated as objects
- Associations are instantiated as links (including aggregation / navigability)
- Links have the **same number of ends as associations**
- **Multiplicities must be respected** (might mean multiple links are required)

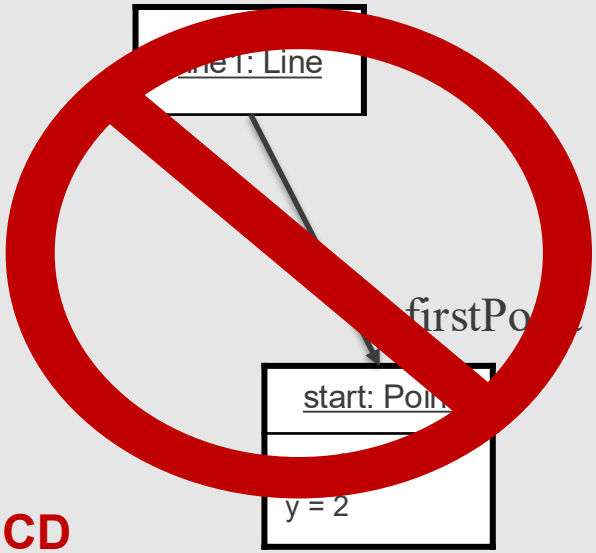
- Association role names can appear on links

First Example



Class diagram

Object diagram

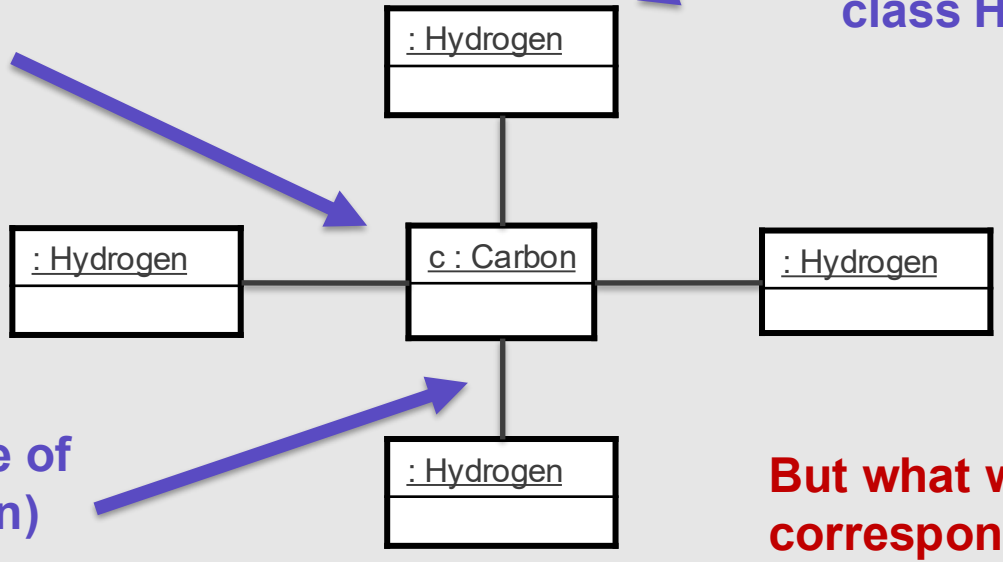


Not valid! All CD constraints must be satisfied

Another Example

Anonymous (unnamed) instance of the class Hydrogen

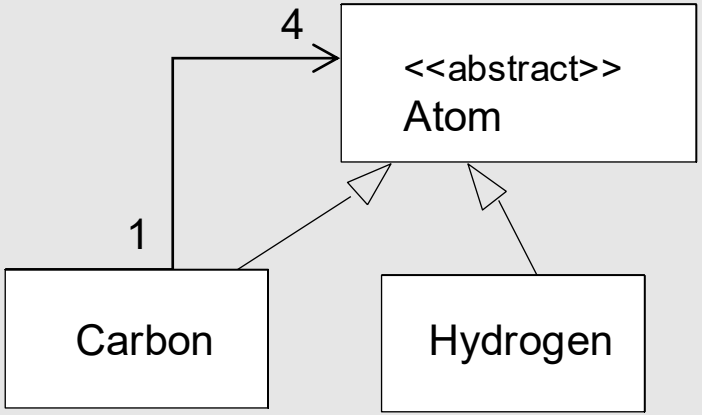
Object named "c" of class Carbon



Link (instance of an association)

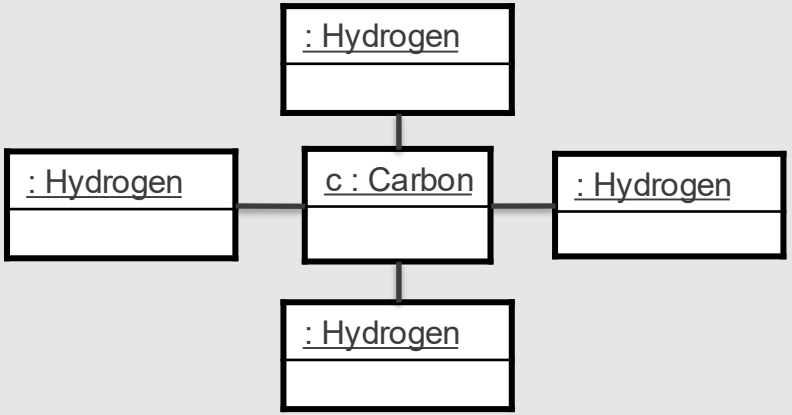
But what would the corresponding class diagram be?

CD – OD Consistency



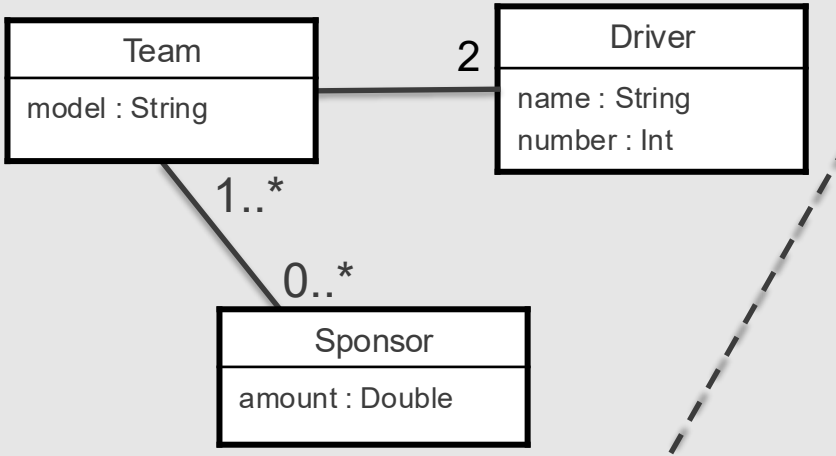
Class diagram

Object diagram

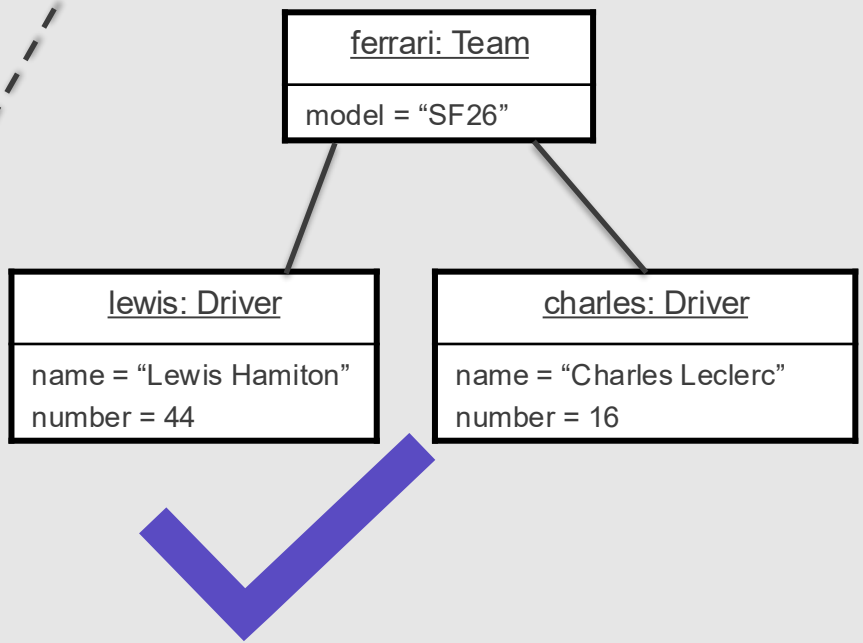


Final Example

Class diagram

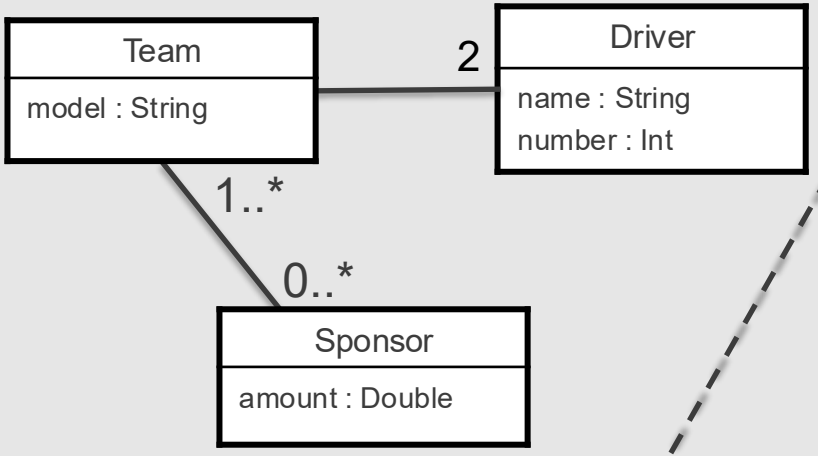


Object diagram

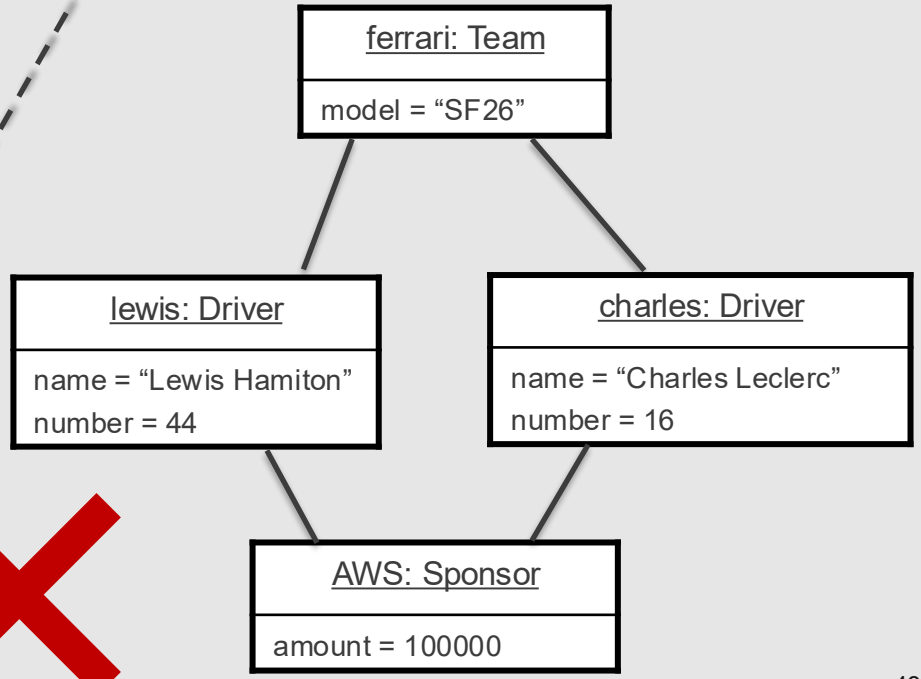


Final Example

Class diagram

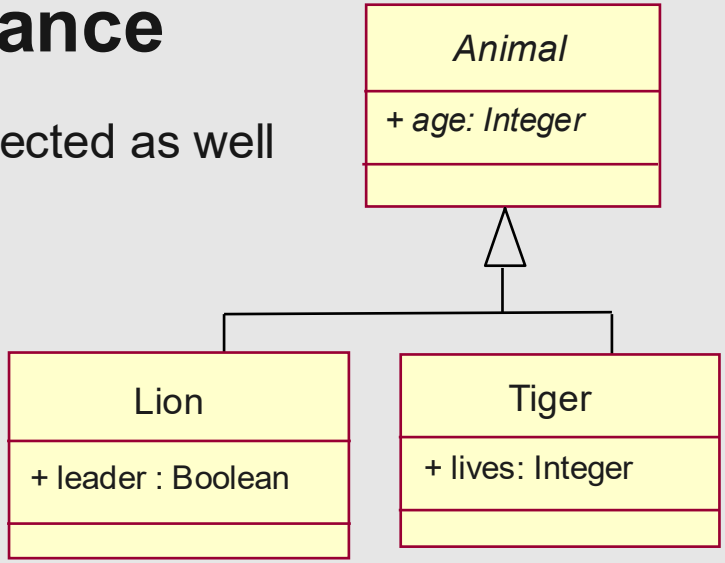


Object diagram

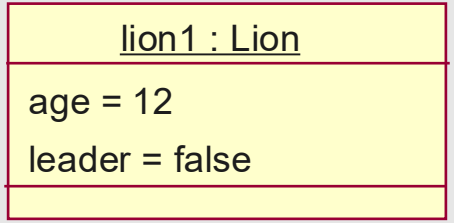


Final consideration: Inheritance

- Consistency requires inheritance to be respected as well
- Abstract classes cannot be instantiated (so no OD can have a: Animal)
- All attributes of subclass need attributes from *Animal*
- Inheritance arrow does NOT appear in OD



- **An example valid Object**



Summary and Next time…

- Describe what an object diagram is, and how we use it
- Explain how to create an object diagram from a corresponding class diagram
- Understand what “consistency” means in this context

Next time: Use Case model tutorial (please bring pen/paper (or a laptop) with you)